PlayHub: Complete Project Guide

This guide explains every file in the project, what each block of code does, why it was written that way, and how each feature works from start to finish.

Table of Contents

1. How the whole system fits together
2. Project file tree
3. Glossary of technologies
4. File-by-file walkthrough
    - 4.1 `.env` and `.env.example`
    - 4.2 `.gitignore`
    - 4.3 `requirements.txt`
    - 4.4 `app.py`
    - 4.5 `schema.sql`
    - 4.6 `templates/base.html`
    - 4.7 `templates/index.html`
    - 4.8 `static/css/style.css`
    - 4.9 `static/js/supabase-client.js`
    - 4.10 `static/js/auth.js`
    - 4.11 `static/js/game.js`
    - 4.12 `build_static.py`
    - 4.13 `setup_deploy.py`
5. Feature-by-feature explanation
6. What happens step by step for each action
7. Design decisions and why
8. Known limitations
9. Troubleshooting: every error we hit and its cause
10. Command reference

---

1. How the whole system fits together

PlayHub has three layers:

Layer	Where it runs	Job
Supabase	Supabase's cloud servers	Stores user accounts, passwords (encrypted), and the `profiles` table with scores. Enforces who can read or change what.
Frontend	The user's browser	The login form, the game, the sidebar. Written in HTML, CSS, and JavaScript.
Flask server	Your computer locally, or Render	Serves the HTML page and injects your Supabase URL and key into it. Only used when running locally or on Render.

Important: once the page is in the browser, the browser talks to Supabase **directly**. Flask is not involved in login, signup, the game, or saving scores. This is why the same site can later be published as plain files on GitHub Pages.

---

2. Project file tree

```
playhub/
├── app.py                      Flask server (Python)
├── build_static.py             Builds plain HTML copy for GitHub Pages
├── setup_deploy.py             Setup helper: folders, config files, git
├── requirements.txt            Python packages to install
├── schema.sql                  Database setup (run once in Supabase)
├── .env                        Your secret keys (never upload)
├── .env.example                Template for .env (safe to upload)
├── .gitignore                  Tells Git which files to skip
├── templates/
│   ├── base.html               Shared page layout
│   └── index.html              Login page and game page content
├── static/
│   ├── css/
│   │   └── style.css           All styling
│   └── js/
│       ├── supabase-client.js  Creates the Supabase connection
│       ├── auth.js             Login, signup, logout, sessions
│       └── game.js             Game, saving scores, leaderboard
├── docs/                       Created by build_static.py (GitHub Pages)
│   ├── index.html
│   ├── .nojekyll
│   └── static/...              Copy of static/
└── venv/                       Python virtual environment (never upload)
```

3. Glossary of technologies

Term	What it is	Why we use it
HTML	The structure of a web page (headings, forms, buttons).	Every web page needs it.
CSS	Rules that control colors, layout, spacing.	Makes the page look like the design.
JavaScript	Code that runs in the browser and makes pages interactive.	Handles clicks, the timer, and talking to Supabase.
Python	The language of the server script.	Flask is written in Python.
Flask	A small Python library that serves web pages.	Lightweight way to serve `index.html` locally.
Jinja2	A template engine. It lets HTML contain placeholders like `{{ value }}` and `{% block %}`.	Flask uses it to fill in the Supabase keys and share one layout across pages.
gunicorn	A production server that runs Flask on the internet.	Render needs it because Flask's built-in server is for development only.
python-dotenv	A library that reads the `.env` file.	Keeps keys out of the code.
Supabase	A hosted backend with a database, login system, and API.	Gives us real accounts and storage without building a server ourselves.
Supabase Auth	The part of Supabase that handles signup, login, passwords, and sessions.	We never store or check passwords ourselves.
Row Level Security (RLS)	Database rules that decide which rows each user can read or change.	Protects data even though the anon key is public.
Anon key	A public key that identifies your project to Supabase.	Required by the browser library. Safe to expose because RLS controls the data.
CDN	A public server that delivers library files (here, `supabase-js`).	Saves us from installing or hosting the library ourselves.
Virtual environment (venv)	A private folder of Python packages for one project.	Keeps this project's packages separate from the rest of your computer.

Term	What it is	Why we use it
Git	A tool that tracks changes to files.	Lets us save versions and send them to GitHub.
GitHub	A website that hosts Git repositories.	Stores the code online.
GitHub Pages	Free hosting for static files from a GitHub repository.	Publishes the `docs` folder as a live website.
Jekyll	A static site generator GitHub runs by default.	We disable it with `.nojekyll` because we don't use it.
Render	A hosting service that runs Python apps.	Alternative host. Requires a card for the free plan now.
Environment variables	Named values stored outside the code.	Render uses them to receive the Supabase keys safely.

---

4. File-by-file walkthrough

4.1 `.env` and `.env.example`

`.env.example` is a template. It shows the format but contains no real keys, so it's safe to upload.

```text
SUPABASE_URL=https://YOUR-PROJECT.supabase.co
SUPABASE_ANON_KEY=YOUR-ANON-PUBLIC-KEY
```

`.env` holds your real values. The line you fixed earlier was:

- `SUPABASE_URL` must be only the project address, for example `https://xenyujspeynnewpnzecw.supabase.co`. Adding `/rest/v1/` broke the connection because the Supabase browser library adds that path itself.
- `SUPABASE_ANON_KEY` is the public key from **Project Settings → API**.

Why separate files: keys must never be pasted into code that gets uploaded. Putting them in `.env` and listing `.env` in `.gitignore` keeps them off GitHub.

4.2 `.gitignore`

```text
```

```
venv/
.env
__pycache__/
*.pyc
```

Each line is a pattern. Git skips any file matching it.

Line	Meaning	Why
`venv/`	The whole venv folder	It's large and specific to your computer. Render installs its own.
`.env`	Your secret keys	Keeps keys off GitHub.
`__pycache__/`	Python's compiled cache folder	Generated automatically, not source code.
`*.pyc`	Compiled Python files	Same reason.

4.3 `requirements.txt`

text

```
flask
python-dotenv
gunicorn
```

Each line is a package that `pip install -r requirements.txt` installs.

- **`flask`**: the web framework that `app.py` uses.
- **`python-dotenv`**: provides `load_dotenv()` in `app.py`.
- **`gunicorn`**: the server Render runs with `gunicorn app:app`.

Note: `build_static.py` also imports `jinja2`. Flask installs Jinja2 automatically as a dependency, so it doesn't need its own line.

4.4 `app.py`

This file is the local web server. Walkthrough line by line:

python

```
import os
```

Imports Python's built-in module for reading environment variables. Used by `os.getenv`.

```
python

from dotenv import load_dotenv
```

Imports the function that reads the `.env` file.

```
python

from flask import Flask, render_template
```

- `Flask` creates the web application.
- `render_template` loads an HTML file from the `templates` folder and fills in its placeholders.

```
python

load_dotenv()
```

Reads `.env` and copies each line into the environment, so `os.getenv` can find it. If no `.env` file exists (for example on Render), this does nothing and returns `False`. That's fine because Render provides the variables directly.

```
python

app = Flask(__name__)
```

Creates the app. `__name__` tells Flask where this file lives, so Flask knows to look for `templates/` and `static/` next to it.

```
python

@app.route("/")
def index():
```

- `@app.route("/")` is a decorator. It tells Flask: when someone requests the path `/` (the home page), run the function directly below.
- `def index():` is the function name. Its name doesn't matter to the browser.

```
python
```

```
    return render_template(
        "index.html",
        supabase_url=os.getenv("SUPABASE_URL", ""),
        supabase_key=os.getenv("SUPABASE_ANON_KEY", ""),
    )
```

- `render_template("index.html", ...)` loads `templates/index.html`.
- `supabase_url=` and `supabase_key=` are variables that the template can use as `{{ supabase_url }}` and `{{ supabase_key }}`.
- `os.getenv("SUPABASE_URL", "")` reads the value. The second argument `""` is the fallback if the variable is missing, so the app doesn't crash.

python

```
if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

- `if __name__ == "__main__":` is true only when you run `python app.py` directly. When gunicorn imports the file on Render, this block is skipped, which is correct.
- `app.run(...)` starts the development server.
- `debug=True` reloads the server automatically when you save a Python file, and shows detailed error pages.
- `port=5000` sets the address to `http://127.0.0.1:5000`.

4.5 `schema.sql`

This file is run once in Supabase's **SQL Editor**. It doesn't run on your computer. It has three parts.

Part 1: the profiles table

sql

```
create table if not exists public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username text unique not null,
  points int not null default 0,
  games_played int not null default 0,
  best_score int not null default 0,
  created_at timestamptz not null default now()
);
```

Part	Meaning	Why
`create table if not exists`	Make the table, but don't fail if it already exists.	Lets you run the file again safely.
`public.profiles`	The table lives in the `public` schema (the default area that the API exposes).	Required for the browser to access it.
`id uuid primary key`	A unique ID per row, in UUID format. Primary key means it must be unique and not empty.	Each profile needs a unique identity.
`references auth.users(id)`	This ID must match a real user in Supabase's login table. This is a foreign key.	Ties each profile to a real account.
`on delete cascade`	If the user is deleted, delete their profile too.	Prevents orphan profiles.
`username text unique not null`	A username string. `unique` prevents two users having the same name. `not null` forces a value.	Usernames are shown in the sidebar and must be distinct.
`points int not null default 0`	Total points. Starts at 0.	Used for the leaderboard ranking.
`games_played int ... default 0`	Number of rounds finished.	Shown on the stats card.
`best_score int ... default 0`	Highest single-round score.	Shown on the stats card.
`created_at timestamptz ... default now()`	Time the row was created, with time zone.	Useful record. Not displayed yet.

Part 2: Row Level Security

```sql
alter table public.profiles enable row level security;
```

Turns on RLS. Once enabled, **no one can read or write the table unless a policy allows it**. Without this line, the public key could read everything.

```sql
create policy "Logged-in users can view all profiles"
  on public.profiles for select
  to authenticated
  using (true);
```

- `for select` applies to reading.
- `to authenticated` means only logged-in users (not anonymous visitors).
- `using (true)` means every row is visible. This is required so the sidebar can show all players.

```sql
create policy "Users can update their own profile"
  on public.profiles for update
  to authenticated
  using (auth.uid() = id)
  with check (auth.uid() = id);
```

- `for update` applies to changing rows.
- `auth.uid()` returns the ID of the logged-in user.
- `using (auth.uid() = id)` means a user can only select rows where the ID matches their own, so they can only update their own row.
- `with check` checks the new values after an update, so a user can't change their row's ID to someone else's.

Notice there is **no insert policy**. Profiles are created only by the trigger below, which runs with elevated rights. Users can't create fake profiles.

Part 3: the automatic profile trigger

```sql
```

```
create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer set search_path = public
as $$
begin
  insert into public.profiles (id, username)
  values (
    new.id,
    coalesce(new.raw_user_meta_data->>'username', split_part(new.email, '@', 1))
  );
  return new;
end;
$$;
```

- `create or replace function` creates a reusable function.
- `returns trigger` means it runs automatically in response to a database event.
- `language plpgsql` is Postgres's programming language for functions.
- `security definer` makes the function run with the creator's permissions, which is needed because a new user doesn't yet have permission to insert into `profiles`.
- `set search_path = public` locks which schema names resolve, which is a security best practice.
- `new.id` is the ID of the user who just signed up.
- `new.raw_user_meta_data->>'username'` reads the username sent during signup. `->>` extracts it as text.
- `coalesce(a, b)` returns `a` if it's not null, otherwise `b`. The fallback `split_part(new.email, '@', 1)` takes the part of the email before the `@`.
- `return new;` ends the function and returns the new row.

sql

```
drop trigger if exists on_auth_user_created on auth.users;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

- The first line removes an old copy of the trigger so the file can be re-run.
- `after insert on auth.users` means: after Supabase creates a user, run the function.
- `for each row` means it runs once per new user.

4.6 `templates/base.html`

This is the shared layout. Other pages extend it.

```html
<!DOCTYPE html>
```

Tells the browser this is an HTML5 document.

```html
<html lang="en">
```

Declares the page's language. Helps screen readers and search engines.

```html
<meta charset="UTF-8">
```

Sets the text encoding so symbols and non-English characters display correctly.

```html
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Makes the page scale to the phone's screen width. Without it, phones show a shrunken desktop layout.

```html
<title>{% block title %}PlayHub{% endblock %}</title>
```

- `{% block title %}` is a placeholder that child pages can replace.
- `PlayHub` is the default title if a child doesn't replace it.
- This is Jinja syntax. It is processed on the server before the browser sees the page.

```html
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

Loads the Supabase JavaScript library from jsDelivr, a public CDN.

- `@supabase/supabase-js` is the package name.
- `@2` means "the latest version 2.x". This prevents a future breaking change (version 3) from affecting the site.

- It's in the `<head>` so the library is ready before any other script uses it.

```html
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
```

- `url_for('static', filename='css/style.css')` is a Flask function. It builds the correct URL, `/static/css/style.css`.
- Using `url_for` instead of typing the path means the URL stays correct if the app is moved to a different folder or host.

```html
<body>
  {% block content %}{% endblock %}
```

The body. `{% block content %}` is where `index.html`'s content gets inserted.

```html
<script>
  window.APP_CONFIG = {
    supabaseUrl: "{{ supabase_url }}",
    supabaseKey: "{{ supabase_key }}"
  };
</script>
```

- This is the bridge between the server and the browser. Flask replaces `{{ supabase_url }}` and `{{ supabase_key }}` with real values.
- `window.APP_CONFIG` creates a global object in the browser. Other scripts read `window.APP_CONFIG.supabaseUrl` to connect.
- Jinja escapes these values automatically, so special characters can't break the script.

```html
  {% block scripts %}{% endblock %}
</body>
```

A placeholder for scripts that child pages add. It's placed at the end of the body so the HTML elements exist before scripts run.

4.7 `templates/index.html`

```
html
{% extends "base.html" %}
```

Says: "use the layout from `base.html`". Everything in this file fills the blocks of that layout.

```
html
{% block content %}
```

Starts the content that goes into `base.html`'s content block.

Login screen

```
html
<div id="auth-screen">
```

A container with an ID so JavaScript can find and show/hide it.

```
html
<div class="auth-card">
  <div class="logo">PlayHub</div>
  <p class="tagline" id="tagline">Log in to start playing</p>
```

- The card is the white-on-dark box in the middle of the screen.
- `.logo` gives the gradient title.
- `id="tagline"` lets JavaScript change the text between login and signup modes.

```
html
<form id="auth-form">
```

A form groups the inputs. Its `submit` event is handled in `auth.js`.

```
html
<input id="username" type="text" placeholder="Username" maxlength="20" class="hidden">
```

- `class="hidden"` hides the field at the start, because the page opens in login mode.
- `maxlength="20"` stops usernames longer than 20 characters in the browser.

- JavaScript removes `hidden` when switching to signup.

```html
<input id="email" type="email" placeholder="Email" required autocomplete="email">
```

- `type="email"` makes mobile keyboards show `@`, and the browser checks the format.
- `required` blocks submission if the field is empty.
- `autocomplete="email"` lets the browser suggest saved emails.

```html
<input id="password" type="password" placeholder="Password" required minlength="6" autoco
```

- `type="password"` hides the characters.
- `minlength="6"` matches Supabase's default minimum password length.
- `autocomplete="current-password"` tells password managers this is a login. `auth.js` changes it to `new-password` during signup.

```html
<button id="auth-btn" type="submit">Log in</button>
```

- `type="submit"` makes this button submit the form when clicked or when Enter is pressed.
- The text starts as "Log in" and changes to "Sign up" in `auth.js`.

```html
<div class="msg" id="auth-msg"></div>
```

An empty area for error or success messages. JavaScript writes into it.

```html
<button class="switch" id="switch-btn" type="button">Don't have an account? Sign up</butt
```

- `type="button"` is important. Without it, the button would submit the form.
- It toggles between login and signup in `auth.js`.

Game screen

```html
```

```
<div id="app-screen" class="hidden">
```

Starts hidden. Shown only after login.

```html
<header>
  <div class="logo">PlayHub</div>
  <div class="header-right">
    <span id="welcome"></span>
    <button id="logout-btn" type="button">Log out</button>
  </div>
</header>
```

- The top bar.
- `#welcome` is filled with `@username` by `game.js`.
- The logout button is handled in `auth.js`.

```html
<div class="stat"><span>Total points</span><strong id="my-points">0</strong></div>
```

Each stat card has a label and a value. `game.js` updates the `strong` element by its ID. The three values are total points, games played, and best round.

```html
<div class="game-box" id="game-box">
  <div class="overlay" id="overlay">
    <h2 id="overlay-title">Tap the Star</h2>
    <p id="overlay-text">You have 15 seconds. Each hit scores one point.</p>
    <button id="start-btn" type="button">Start round</button>
  </div>
</div>
```

- `#game-box` is the play area. Targets are placed inside it.
- `.overlay` is the dark panel that covers the play area before and after a round. `game.js` hides it during play.
- The title, text, and button are updated for the start screen and the results screen.

```html
```

```
<div class="game-controls">
  <span>Time: <strong id="time">15</strong>s</span>
  <span>Score: <strong id="score">0</strong></span>
</div>
```

The countdown and the score for the current round.

html

```
<aside>
  <h3>Players progress</h3>
  <div id="leaderboard"></div>
</aside>
```

- `<aside>` is the sidebar. Semantically it means "supporting content".
- `#leaderboard` is empty until `game.js` fills it with one row per player.

html

```
{% endblock %}
```

Ends the content block.

Script loading

html

```
{% block scripts %}
<script src="{{ url_for('static', filename='js/supabase-client.js') }}"></script>
<script src="{{ url_for('static', filename='js/auth.js') }}"></script>
<script src="{{ url_for('static', filename='js/game.js') }}"></script>
{% endblock %}
```

- The scripts load in this order on purpose. `supabase-client.js` creates `window.sb`, which `auth.js` and `game.js` both need.
- `url_for` builds the paths, the same as in `base.html`.

4.8 `static/css/style.css`

Variables

css

```css
:root {
  --bg: #0b0b14;
  --card: #14141f;
  --border: #26263a;
  --text: #f2f2f7;
  --muted: #8b8ba7;
  --accent: #ff3cac;
  --accent2: #784ba0;
  --accent3: #2b86c5;
}
```

- `:root` is the top-level element. Variables declared here are available everywhere.
- `var(--accent)` reads a value. Changing one line here changes the color across the entire site.

Reset and base

```css
* { box-sizing: border-box; margin: 0; padding: 0; }
```

- `*` selects every element.
- `box-sizing: border-box` makes width include padding, so layouts don't overflow unexpectedly.
- `margin: 0; padding: 0` removes browser default spacing so all layouts start from the same baseline.

```css
body {
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  background: radial-gradient(circle at top, #1d1033, var(--bg) 60%);
  color: var(--text);
  min-height: 100vh;
}
```

- The font list uses the system's native font first, so the page looks at home on each device.
- `radial-gradient` creates the soft purple glow at the top.
- `min-height: 100vh` makes the background cover the full screen height.

```css
.hidden { display: none !important; }
```

The single rule that hides screens and fields. `!important` ensures nothing else can override it, for example a `display: flex` set on a container.

Buttons and inputs

```css
button { ... background: linear-gradient(90deg, var(--accent), var(--accent2)); ... }
button:hover { opacity: 0.9; }
button:active { transform: scale(0.97); }
button:disabled { opacity: 0.5; cursor: not-allowed; }
```

- The default button style applies to every button.
- `:hover` slightly dims the button when the mouse is over it.
- `:active` shrinks the button briefly when clicked, giving tactile feedback.
- `:disabled` shows the button is unavailable during a request, so users don't click twice.
- `transition` animates changes smoothly over the given time.

```css
input { ... }
input:focus { border-color: var(--accent); }
```

- Inputs share a dark style.
- `:focus` highlights the field the user is typing in.

```css
.logo {
  ...
  background: linear-gradient(90deg, var(--accent), var(--accent2), var(--accent3));
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
```

This creates gradient text:

1. The gradient is painted as the element's background.
2. `background-clip: text` restricts the background to the letter shapes.
3. `color: transparent` hides the normal text color so the gradient shows through.
4. `-webkit-` is the prefix older Safari and Chrome versions need.

Auth screen

```css
#auth-screen { display: flex; flex-direction: column; align-items: center; justify-conten
```

A flex container that centers the card horizontally (`align-items`) and vertically (`justify-content`).

```css
.auth-card { ... max-width: 350px; text-align: center; }
```

The card has a fixed maximum width, so it doesn't stretch across wide screens.

```css
#auth-form { display: flex; flex-direction: column; gap: 10px; }
```

Stacks the inputs and button vertically with 10px between them.

```css
.switch { background: none; color: var(--accent); ... }
```

Overrides the default gradient button so the switch link looks like plain text.

```css
.msg { ... color: #ff6b8a; }
.msg.ok { color: #5be39a; }
```

Error messages are pink-red. The `.ok` class (added in `auth.js`) turns success messages green.

App screen layout

```css
#app-screen { display: grid; grid-template-columns: 1fr 280px; min-height: 100vh; }
```

A two-column grid. The main area takes all remaining space (`1fr`), and the sidebar is fixed at 280px.

```css
header { display: flex; justify-content: space-between; align-items: center; flex-wrap: w
```

The top bar puts the logo on the left and the user info on the right. `flex-wrap` lets items drop to a new line on narrow screens.

```css
.stats { display: flex; gap: 12px; margin-bottom: 20px; flex-wrap: wrap; }
.stat { ... min-width: 130px; }
```

Stat cards sit in a row and wrap if there isn't room.

Game area

```css
.game-box { position: relative; width: 100%; max-width: 560px; height: 340px; margin: 0 a
```

- `position: relative` makes this the reference point for absolutely positioned children (the targets and the overlay).
- `max-width` caps its size on large screens.
- `margin: 0 auto` centers it.
- `overflow: hidden` clips anything that would stick out, so targets can't leak outside.

```css
.target {
  position: absolute;
  width: 64px; height: 64px;
  border-radius: 50%;
  background: radial-gradient(circle at 30% 30%, #fff, var(--accent));
  box-shadow: 0 0 24px var(--accent);
  transition: transform 0.08s;
}
.target:active { transform: scale(0.85); }
```

- `position: absolute` lets `game.js` place the target at exact pixel coordinates.
- `border-radius: 50%` makes it a circle.
- The radial gradient's highlight at 30% 30% gives a 3D-ish look.
- `box-shadow` creates the glow.
- `:active` shrinks it briefly on click.

```css
```

```css
.overlay {
  position: absolute;
  inset: 0;
  display: flex; flex-direction: column; align-items: center; justify-content: center;
  background: rgba(11, 11, 20, 0.8);
}
```

- `inset: 0` stretches it to cover the whole game box (shorthand for top, right, bottom, and left all 0).
- The semi-transparent background darkens the game behind it.

```css
.game-controls { display: flex; justify-content: center; gap: 24px; margin-top: 16px; }
```

Centers the timer and score side by side.

Sidebar

```css
aside {
  ...
  position: sticky;
  top: 0;
  max-height: 100vh;
  overflow-y: auto;
}
```

- `sticky` keeps the sidebar visible while scrolling the main page.
- `overflow-y: auto` adds a scrollbar if there are many players.

```css
.player { display: flex; align-items: center; gap: 10px; ... }
.player.me { border-color: var(--accent); }
```

Each leaderboard row is a flex row. `.me` is added to the current user's row to highlight it.

```css
```

```
.pinfo { flex: 1; min-width: 0; }
.pname { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
```

- `flex: 1` makes the info block take the remaining width.
- `min-width: 0` lets it shrink below its content size, which is needed for ellipsis to work.
- The three properties on `.pname` cut long usernames with "..." instead of wrapping.

```css
.bar { height: 4px; ... }
.bar div { height: 100%; background: linear-gradient(...); }
```

The progress bar. The outer track is grey, and the inner fill's width is set by JavaScript as a percentage.

Responsive layout

```css
@media (max-width: 760px) {
  #app-screen { grid-template-columns: 1fr; }
  aside { border-left: none; border-top: 1px solid var(--border); position: static; max-h
}
```

On screens narrower than 760px, the two columns become one, and the sidebar moves below the game.

4.9 `static/js/supabase-client.js`

```javascript
(() => {
  const { supabaseUrl, supabaseKey } = window.APP_CONFIG;
  window.sb = supabase.createClient(supabaseUrl, supabaseKey);
})();
```

Line	What it does	Why
`(() => { ... })();`	An immediately invoked function expression. It runs right away and keeps variables inside it private.	Prevents variable names from clashing with other scripts.
`const { supabaseUrl, supabaseKey } = window.APP_CONFIG;`	Destructuring: pulls two properties out of the config object into variables.	Shorter than writing `window.APP_CONFIG.supabaseUrl` each time.
`supabase.createClient(...)`	Creates the connection to your Supabase project. `supabase` is the global object the CDN script provides.	This is the only place the connection is made.
`window.sb = ...`	Stores the client on the global `window` object.	`auth.js` and `game.js` need the same client, and this is how they share it.

4.10 `static/js/auth.js`

Wrapped in the same `(() => { ... })();` pattern. Walkthrough:

Setup

```javascript
const sb = window.sb;
const $ = id => document.getElementById(id);
```

- `const sb = window.sb;` takes the client created by the previous script.
- `$` is a short helper. `$('email')` is the same as `document.getElementById('email')`. It saves typing.

```javascript
let currentUser = null;
let mode = 'login';
```

- `currentUser` remembers who is logged in. `null` means nobody.
- `mode` is either `'login'` or `'signup'`, and decides what the form does on submit.

Switching between login and signup

```javascript
$('switch-btn').onclick = () => {
  mode = mode === 'login' ? 'signup' : 'login';
  const signup = mode === 'signup';
```

- Flips the mode. The ternary `? :` is shorthand for if/else.
- `signup` is a true/false flag used below.

```javascript
  $('username').classList.toggle('hidden', !signup);
```

`classList.toggle(name, force)` adds the class when `force` is true and removes it when false. Here, `hidden` is added when *not* signing up, so the username field appears only in signup mode.

```javascript
  $('username').required = signup;
  $('password').autocomplete = signup ? 'new-password' : 'current-password';
```

- The username is required only in signup.
- `new-password` tells password managers to offer a strong generated password.

```javascript
  $('auth-btn').textContent = signup ? 'Sign up' : 'Log in';
  $('tagline').textContent = ...;
  $('switch-btn').textContent = ...;
```

Updates the wording on the card to match the mode.

```javascript
  $('auth-msg').textContent = '';
  $('auth-msg').className = 'msg';
};
```

Clears any old error message and resets the message style (removes `ok` if it was set).

Submitting the form

```javascript
$('auth-form').onsubmit = async e => {
  e.preventDefault();
```

- `onsubmit` runs when the form is submitted.
- `async` allows `await` inside, so the code can wait for Supabase.
- `e.preventDefault()` stops the browser's default action, which is reloading the page.

```javascript
  const btn = $('auth-btn');
  const msg = $('auth-msg');
  const email = $('email').value.trim();
  const password = $('password').value;
```

- `.value` reads what the user typed.
- `.trim()` removes accidental spaces around the email, which would otherwise cause a login failure.
- The password is not trimmed, because spaces can be part of a real password.

```javascript
  msg.className = 'msg';
  btn.disabled = true;
```

Clears the old message style and disables the button so the user can't submit twice while waiting.

```javascript
  try {
    if (mode === 'signup') {
      const username = $('username').value.trim();
      if (username.length < 3) throw new Error('Username must be at least 3 characters.')
```

- `try` lets us catch any error from Supabase.
- Client-side check: usernames must be at least 3 characters. This gives instant feedback before contacting Supabase.
- `throw new Error(...)` jumps straight to the `catch` block, which shows the message.

```javascript
```

```
    const { data, error } = await sb.auth.signUp({
      email,
      password,
      options: { data: { username } }
    });
    if (error) throw error;
```

- `sb.auth.signUp` creates the account in Supabase.
- `email,` and `password,` use shorthand for `email: email`.
- `options: { data: { username } }` stores the username in the user's metadata. The database trigger in `schema.sql` reads it from there.
- `await` pauses until Supabase responds.
- Supabase returns `{ data, error }`. If `error` exists, we throw it so the `catch` block handles it.

javascript

```
    if (!data.session) {
      msg.classList.add('ok');
      msg.textContent = 'Account created! Check your email to confirm, then log in.';
      $('switch-btn').click();
    }
```

- If email confirmation is on, Supabase creates the account but **doesn't** return a session. `!data.session` detects this.
- The message is shown in green (`ok`).
- `$('switch-btn').click()` programmatically clicks the switch button, returning the form to login mode.

javascript

```
  } else {
    const { error } = await sb.auth.signInWithPassword({ email, password });
    if (error) throw error;
  }
```

Login. `signInWithPassword` checks the password with Supabase and, if correct, creates a session. Nothing else is needed here, because `onAuthStateChange` (below) reacts to the new session.

javascript

```
  } catch (err) {
    msg.textContent = err.message;
  } finally {
    btn.disabled = false;
  }
};
```

- `catch` shows the error text under the form (for example, "Invalid login credentials").
- `finally` runs whether it succeeded or failed, re-enabling the button.

Logout

javascript

```
$('logout-btn').onclick = async () => {
  await sb.auth.signOut();
};
```

`signOut` deletes the session from the browser and from Supabase. The screen change happens in `onAuthStateChange`.

Session handling (the most important part)

javascript

```
sb.auth.onAuthStateChange((event, session) => {
```

Supabase calls this function whenever the login state changes. It is called on page load (`INITIAL_SESSION`), on login, on logout, and when the access token refreshes in the background.

javascript

```
  if (session) {
    if (!currentUser || currentUser.id !== session.user.id) {
      currentUser = session.user;
      showApp();
      document.dispatchEvent(new CustomEvent('playhub:login', { detail: currentUser }));
    }
```

- `if (session)` means someone is logged in.
- `!currentUser || currentUser.id !== session.user.id` is true only when this is a *new* login. The second check stops background token refreshes from re-running the setup and resetting the game mid-round.

- `showApp()` hides the login form and shows the game.
- `document.dispatchEvent(new CustomEvent(...))` broadcasts a message named `playhub:login`. `game.js` listens for it and loads the player's data. `detail` carries the user object.

```javascript
  } else if (currentUser) {
    currentUser = null;
    showAuth();
    document.dispatchEvent(new Event('playhub:logout'));
  }
});
```

- `else if (currentUser)` means: there is no session, but we thought someone was logged in. That's a logout.
- `showAuth()` shows the login form.
- The `playhub:logout` event tells `game.js` to stop the timer.

Screen switching

```javascript
function showApp() {
  $('auth-screen').classList.add('hidden');
  $('app-screen').classList.remove('hidden');
}

function showAuth() {
  $('app-screen').classList.add('hidden');
  $('auth-screen').classList.remove('hidden');
  $('email').value = '';
  $('password').value = '';
}
```

- Each function swaps the `hidden` class between the two screens.
- `showAuth` clears the email and password fields so the next person on a shared computer can't see them.

Why you can't log in again while logged in: the login form is hidden whenever a session exists. Supabase stores the session in the browser's local storage, so refreshing the page restores it, and `onAuthStateChange` immediately shows the game.

4.11 `static/js/game.js`

Setup

```javascript
const sb = window.sb;
const $ = id => document.getElementById(id);

const ROUND_SECONDS = 15;
const TARGET_SIZE = 64;
```

- `ROUND_SECONDS` and `TARGET_SIZE` are constants. Change them here to adjust the game. `TARGET_SIZE` must match the CSS width and height of `.target` (64px).

```javascript
let currentUser = null;
let score = 0;
let timeLeft = ROUND_SECONDS;
let timer = null;
let roundActive = false;
```

- `currentUser` is this file's own copy of the logged-in user. It is set from the event, not read from `auth.js`.
- `score` counts hits in the current round.
- `timeLeft` counts down.
- `timer` holds the interval ID so it can be cleared.
- `roundActive` prevents hits from counting after the round has ended.

Reacting to login and logout

```javascript
document.addEventListener('playhub:login', async e => {
  currentUser = e.detail;
  resetGame();
  await refreshAll();
});
```

- Listens for the event sent by `auth.js`.
- `e.detail` is the user object passed with the event.
- `resetGame()` prepares the start screen.

- `refreshAll()` loads the profile and leaderboard from Supabase.

```javascript
document.addEventListener('playhub:logout', () => {
  currentUser = null;
  clearInterval(timer);
  roundActive = false;
});
```

Stops the timer so the countdown doesn't keep running after logout.

Loading the player's own profile

```javascript
async function loadMyProfile() {
  const { data, error } = await sb.from('profiles')
    .select('username, points, games_played, best_score')
    .eq('id', currentUser.id)
    .single();
  if (error || !data) return;
```

- `sb.from('profiles')` targets the table.
- `.select(...)` requests only the four columns the page needs.
- `.eq('id', currentUser.id)` is a WHERE clause: only the row for this user.
- `.single()` expects exactly one row and returns it as an object rather than an array.
- If there's an error or no data, the function stops quietly.

```javascript
  $('welcome').textContent = '@' + data.username;
  $('my-points').textContent = data.points;
  $('my-games').textContent = data.games_played;
  $('my-best').textContent = data.best_score;
}
```

Writes the values into the stat cards. `textContent` is used instead of `innerHTML` so the username can't inject code.

Loading the leaderboard

```javascript
```

```
async function loadLeaderboard() {
  const { data, error } = await sb.from('profiles')
    .select('id, username, points')
    .order('points', { ascending: false })
    .limit(50);
  if (error || !data) return;
```

- Requests every profile's ID, name, and points.
- `.order('points', { ascending: false })` sorts highest first.
- `.limit(50)` caps the result at 50 players to keep the page fast.
- Access is allowed because of the "view all profiles" RLS policy.

javascript

```
  const topPoints = Math.max(...data.map(p => p.points), 1);
```

- `data.map(p => p.points)` makes a list of every player's points.
- `...` spreads the list into `Math.max`, which returns the highest value.
- The `1` is a floor. If everyone has 0 points, the top would be 0, and dividing by zero would break the progress bars. With the floor, the minimum is 1.

javascript

```
  const list = $('leaderboard');
  list.innerHTML = '';
```

Finds the sidebar container and clears the old rows before rebuilding.

javascript

```
  data.forEach((p, i) => {
    const row = document.createElement('div');
    row.className = 'player' + (p.id === currentUser?.id ? ' me' : '');
```

- Loops through each player. `i` is the position (0 for first).
- `document.createElement('div')` builds the element in memory.
- `p.id === currentUser?.id` checks whether this row is the logged-in user. The `?.` (optional chaining) returns `undefined` instead of crashing if `currentUser` is null.
- The `me` class highlights your row.

javascript

```
const rank = document.createElement('div');
rank.className = 'rank';
rank.textContent = i + 1;
```

The position number. `i + 1` makes it start at 1 instead of 0.

javascript

```
const info = document.createElement('div');
info.className = 'pinfo';

const prow = document.createElement('div');
prow.className = 'prow';
const name = document.createElement('span');
name.className = 'pname';
name.textContent = '@' + p.username;
const pts = document.createElement('span');
pts.className = 'pts';
pts.textContent = p.points;
prow.append(name, pts);
```

- Builds the name row. `textContent` displays the username as plain text, so a name like `<img onerror=...>` would show as text instead of running code.
- `prow.append(name, pts)` adds both spans inside the row.

javascript

```
const bar = document.createElement('div');
bar.className = 'bar';
const fill = document.createElement('div');
fill.style.width = (p.points / topPoints * 100) + '%';
bar.appendChild(fill);
```

- The progress bar. Its width is the player's points as a percentage of the top player's points.
- Example: 5 points out of a top score of 10 gives 50%.

javascript

```
    info.append(prow, bar);
    row.append(rank, info);
    list.appendChild(row);
  });
}
```

Assembles the pieces: `info` holds the name row and bar, `row` holds the rank and info, and the finished row is added to the sidebar.

Saving the round

javascript

```
async function saveRound(points) {
  const { data: p, error } = await sb.from('profiles')
    .select('points, games_played, best_score')
    .eq('id', currentUser.id)
    .single();
  if (error || !p) return false;
```

First reads the current totals, because the database has no "add to" command in this setup. The function returns `false` if the read fails.

javascript

```
  const { error: updateError } = await sb.from('profiles').update({
    points: p.points + points,
    games_played: p.games_played + 1,
    best_score: Math.max(p.best_score, points)
  }).eq('id', currentUser.id);
```

- `.update({...})` writes new values.
- `p.points + points` adds this round's score to the total.
- `games_played + 1` counts the round.
- `Math.max(p.best_score, points)` keeps whichever is higher: the old best or this round.
- `.eq('id', ...)` limits the update to this user's row. RLS would also block any other row.

javascript

```
  if (updateError) return false;
  await refreshAll();
  return true;
}
```

If the write succeeded, refresh the screen so the new totals and leaderboard appear, then return `true`.

Known weakness: if two rounds were saved at exactly the same moment from two tabs, one update could overwrite the other. This is unlikely in normal play.

Game setup and reset

javascript

```
function resetGame() {
  score = 0;
  timeLeft = ROUND_SECONDS;
  roundActive = false;
  $('score').textContent = 0;
  $('time').textContent = ROUND_SECONDS;
  document.querySelectorAll('.target').forEach(t => t.remove());
```

- Resets all round variables and the on-screen counters.
- `querySelectorAll('.target')` finds every target element, and `.remove()` deletes each one.

javascript

```
  $('overlay-title').textContent = 'Tap the Star';
  $('overlay-text').textContent = `You have ${ROUND_SECONDS} seconds. Each hit scores one
  $('start-btn').textContent = 'Start round';
  $('start-btn').onclick = startRound;
  $('overlay').classList.remove('hidden');
}
```

- Sets the start-screen wording.
- The backticks with `${...}` are a template string, which inserts the variable into the text.
- `$('start-btn').onclick = startRound;` attaches the start function to the button.
- Removing `hidden` shows the overlay.

Starting a round

javascript

```
function startRound() {
  score = 0;
  timeLeft = ROUND_SECONDS;
  roundActive = true;
  $('score').textContent = 0;
  $('time').textContent = timeLeft;
  $('overlay').classList.add('hidden');
  moveTarget();
```

- Resets the round and marks it active so hits will count.
- Hides the overlay so the game area is clear.
- Places the first target.

javascript

```
  clearInterval(timer);
  timer = setInterval(() => {
    timeLeft--;
    $('time').textContent = timeLeft;
    if (timeLeft <= 0) endRound();
  }, 1000);
}
```

- `clearInterval(timer)` stops any previous timer first, so two timers never run at once.
- `setInterval(fn, 1000)` runs the function every 1000 milliseconds (one second).
- Each tick subtracts 1 from `timeLeft` and updates the display.
- When it reaches 0, `endRound()` runs.

Placing targets

javascript

```
function moveTarget() {
  const box = $('game-box');
  let t = box.querySelector('.target');

  if (!t) {
    t = document.createElement('button');
    t.className = 'target';
    t.type = 'button';
    t.onclick = () => {
      if (!roundActive) return;
      score++;
      $('score').textContent = score;
      moveTarget();
    };
    box.appendChild(t);
  }
```

- Looks for an existing target. If there isn't one, creates a new `<button>`.
- `type = 'button'` prevents accidental form submission.
- The click handler: ignores clicks when no round is active. Otherwise it adds a point, updates the display, and moves the target.
- `box.appendChild(t)` adds it to the game area.

javascript

```
  const x = Math.random() * (box.clientWidth - TARGET_SIZE);
  const y = Math.random() * (box.clientHeight - TARGET_SIZE);
  t.style.left = x + 'px';
  t.style.top = y + 'px';
}
```

- `Math.random()` returns a number from 0 up to (not including) 1.
- Multiplying by `(width - 64)` gives a random horizontal position that keeps the target fully inside the box.
- The same is done vertically.
- Setting `left` and `top` moves the absolutely positioned target there.

Ending a round

javascript

```
async function endRound() {
  clearInterval(timer);
  roundActive = false;
  document.querySelectorAll('.target').forEach(t => t.remove());
```

Stops the timer, blocks further hits, and removes the target.

javascript

```
  $('overlay').classList.remove('hidden');
  $('overlay-title').textContent = 'Round over!';
  $('overlay-text').textContent = `You scored ${score} point${score === 1 ? '' : 's'}. Sa
  $('start-btn').textContent = 'Play again';
  $('start-btn').onclick = startRound;
```

- Shows the results overlay.
- `score === 1 ? '' : 's'` picks "point" or "points" depending on the number.
- The button now starts another round.

javascript

```
  const saved = await saveRound(score);
  $('overlay-text').textContent = saved
    ? `Saved! You scored ${score} point${score === 1 ? '' : 's'}.`
    : 'Could not save your score. Check your connection and try again.';
}
```

- Saves the score and waits for the result.
- Shows either a success message or an error message.

4.12 `build_static.py`

This script creates the GitHub Pages version of the site.

python

```
from dotenv import load_dotenv
from jinja2 import Environment, FileSystemLoader
```

- `load_dotenv` reads `.env`.
- `Environment` and `FileSystemLoader` are Jinja's tools for loading and rendering templates without Flask.

python

```
BASE = Path(__file__).resolve().parent
load_dotenv(BASE / ".env")
```

- `__file__` is this script's path. `.resolve().parent` gives its folder, so the script works no matter where you run it from.
- Loads the `.env` file from that folder.

python

```
supabase_url = os.getenv("SUPABASE_URL", "").strip().rstrip("/")
supabase_url = re.sub(r"/rest/v1$", "", supabase_url)
supabase_key = os.getenv("SUPABASE_ANON_KEY", "").strip()
```

- Reads the values, removes stray spaces, and removes a trailing `/`.
- The regex removes `/rest/v1` from the end if it was added by mistake, the same fix as before.

python

```
if not supabase_url or not supabase_key:
    sys.exit("Missing SUPABASE_URL or SUPABASE_ANON_KEY in your .env file.")
```

Stops the build with a clear message if either value is empty. This prevents publishing a broken site.

python

```
env = Environment(loader=FileSystemLoader(BASE / "templates"), autoescape=True)
env.globals["url_for"] = lambda endpoint, filename="": f"static/{filename}"
```

- Creates a Jinja environment that loads files from `templates/`.
- `autoescape=True` escapes HTML special characters automatically, the same as Flask does.
- Flask's `url_for` doesn't exist here, so we provide a replacement. The lambda returns a relative path like `static/css/style.css`. Relative paths work on GitHub Pages, which serves the site from a subfolder (`/playhub/`).

python

```
html = env.get_template("index.html").render(
    supabase_url=supabase_url,
    supabase_key=supabase_key,
)
```

- Loads `index.html`. Because it extends `base.html`, Jinja loads that too.
- `render(...)` fills in every `{{ }}` and `{% block %}`, producing a finished HTML string.

```python
docs = BASE / "docs"
docs.mkdir(exist_ok=True)
(docs / "index.html").write_text(html, encoding="utf-8")
(docs / ".nojekyll").write_text("", encoding="utf-8")
```

- Creates the `docs` folder if needed.
- Writes the page.
- Creates an empty `.nojekyll` file. Without it, GitHub runs Jekyll, which ignores files and folders starting with an underscore or dot. The file tells GitHub to serve everything as-is.

```python
static_dst = docs / "static"
if static_dst.exists():
    shutil.rmtree(static_dst)
shutil.copytree(BASE / "static", static_dst)
```

- Deletes the old copy of `static` (so removed files don't linger).
- Copies the current `static` folder into `docs`.

```python
print("Built docs/index.html and docs/static/")
print(f"Supabase URL used: {supabase_url}")
```

Confirms what was built and which URL was used, so you can spot mistakes.

4.13 `setup_deploy.py`

A helper that does file setup in one step. It doesn't upload anything.

Function	What it does
`create_folders()`	Makes `templates`, `static/css`, and `static/js` if they're missing. `parents=True` creates parent folders too. `exist_ok=True` avoids errors if they exist.
`write_file(rel_path, content)`	Writes text to a file, creating folders first. `encoding="utf-8"` keeps special characters correct.
`write_project_files()`	Writes `.gitignore` and `requirements.txt`.
`fix_env()`	Reads `.env`, removes `/rest/v1` from `SUPABASE_URL`, and writes it back. It never prints the anon key.
`check_files()`	Checks every required file exists and prints `OK` or `MISSING`.
`run(cmd)`	Runs a command in the project folder. `check=True` raises an error if the command fails.
`setup_git()`	Runs `git init` if needed, `git add .`, `git commit`, and `git branch -M main`. Only runs with `--git`.
`main()`	Calls the steps in order. `sys.argv` checks for the `--git` flag.

Why it doesn't push: connecting to GitHub needs your username and a sign-in, so the script leaves that to you.

---

5. Feature-by-feature explanation

Feature: Sign up

- **Where:** `index.html` (form), `auth.js` (`signUp` call), `schema.sql` (trigger).
- **Flow:** the form collects username, email, and password → `auth.js` checks the username length → Supabase creates the user → the trigger creates the `profiles` row → if no confirmation is needed, the session starts and the game opens.
- **Why:** signup and password storage are handled by Supabase, which is secure by default. We only add the username.

Feature: Log in

- **Where:** `auth.js` (`signInWithPassword`).
- **Flow:** Supabase checks the password → a session is created and stored in the browser → `onAuthStateChange` shows the game.

Feature: Stay logged in across page refreshes

- **Where:** Supabase client (automatic), `auth.js` (`onAuthStateChange`).
- **Flow:** Supabase saves the session in browser storage. On load, the library restores it and fires `INITIAL_SESSION`, and `auth.js` shows the game.

Feature: Cannot sign in again while logged in

- **Where:** `auth.js` (`showApp` hides `auth-screen`, `onAuthStateChange`).
- **Why it works:** the login form is hidden whenever a session exists. There is no way to show it without logging out.

Feature: Log out

- **Where:** `index.html` (`logout-btn`), `auth.js` (`signOut`).
- **Flow:** `signOut` clears the session → `onAuthStateChange` sees no session → `showAuth` shows the login form → `game.js` stops the timer.

Feature: Password rules

- **Where:** `index.html` (`minlength="6"`), Supabase settings.
- **Why:** the browser enforces it first, and Supabase enforces its own minimum too.

Feature: Email confirmation (optional)

- **Where:** `auth.js` (`if (!data.session)`), Supabase setting.
- **Flow:** if confirmation is on, the user gets an email and must click it before logging in. The app shows a message instead of opening the game.
- **Turning it off:** Supabase → Authentication → Sign In / Providers → Email → untick "Confirm email". Useful for testing.

Feature: The game (Tap the Star)

- **Where:** `game.js` (`startRound`, `moveTarget`, `endRound`), `style.css` (`.target`, `.overlay`).
- **Flow:** Start → a 15-second timer begins → each tap adds a point and moves the target → at zero, the round ends.
- **Why it's simple:** no canvas or animation library needed. Plain DOM elements and `setInterval` are enough, and they're easy to read.

Feature: Saving scores

- **Where:** `game.js` (`saveRound`), `schema.sql` (update policy).
- **Flow:** round ends → read current totals → add this round → write back → refresh the screen.
- **Why the policy matters:** RLS means the browser can only update the logged-in user's own row.

Feature: Stats cards

- **Where:** `index.html` (stat elements), `game.js` (`loadMyProfile`).
- **Shows:** total points, games played, and best round, all from the database.

Feature: Leaderboard sidebar

- **Where:** `index.html` (`#leaderboard`), `game.js` (`loadLeaderboard`), `style.css` (sidebar styles).
- **Flow:** query the top 50 profiles sorted by points → build one row per player → highlight yours → draw a progress bar relative to the top player.
- **Why it updates:** it reloads after every saved round.

Feature: Progress bars

- **Where:** `game.js` (`fill.style.width`).
- **How:** each bar is `points ÷ top points × 100%`. The top player's bar is full.

Feature: Highlighting your own row

- **Where:** `game.js` (`p.id === currentUser?.id`), `style.css` (`.player.me`).

Feature: Responsive layout

- **Where:** `style.css` (`@media (max-width: 760px)`), `index.html` (viewport meta tag in `base.html`).
- **How:** on phones, the sidebar moves below the game.

Feature: Safe display of usernames

- **Where:** `game.js` (`textContent` everywhere user data is shown).
- **Why:** `textContent` never interprets text as code, which blocks a common attack.

Feature: Protection against double-submit

- **Where:** `auth.js` (`btn.disabled = true` and `finally`).

Feature: Protection of data

- **Where:** `schema.sql` (RLS policies, the trigger's `security definer`).
- **Why:** even with the public key visible in the page, users can only read profiles and only change their own.

Feature: Local and live versions

- **Local:** `app.py` serves the page with keys from `.env`.
- **Live (GitHub Pages):** `build_static.py` creates `docs/`, which GitHub serves.
- **Live (Render):** the same Flask app, with keys set in Render's dashboard.

---

6. What happens step by step for each action

Opening the page (local):

1. Browser requests `http://127.0.0.1:5000/`.
2. `app.py` runs `index()`, which renders `index.html` with keys.

3. The browser downloads the CSS, the Supabase library, and the three scripts.
4. `supabase-client.js` creates `window.sb`.
5. `auth.js` runs `onAuthStateChange`. If there's no session, the login form stays visible.

Signing up:

1. Form submit → `preventDefault` → button disabled.
2. `signUp` sent to Supabase.
3. Supabase inserts into `auth.users`.
4. Trigger inserts into `profiles`.
5. Response returns. Either the session starts (game opens) or the confirmation message appears.

Logging in:

1. `signInWithPassword` sent to Supabase.
2. Supabase checks the password, returns a session.
3. `onAuthStateChange` fires with the session.
4. `auth.js` shows the game and dispatches `playhub:login`.
5. `game.js` resets the game and loads profile and leaderboard.

Playing a round:

1. Start → `roundActive = true`, timer starts, target appears.
2. Each tap → `score++`, display updates, target moves.
3. Timer reaches 0 → `endRound` → target removed → overlay shows.
4. `saveRound` reads totals, writes new totals, refreshes.

Logging out:

1. `signOut` clears the session.
2. `onAuthStateChange` fires with no session.
3. `auth.js` shows the login form and dispatches `playhub:logout`.
4. `game.js` stops the timer.

Publishing to GitHub Pages:

1. `python build_static.py` renders `docs/index.html` with keys and copies `static/`.
2. `git add`, `commit`, `push` sends `docs/` to GitHub.
3. GitHub Pages serves `docs/` at `https://zzanizzzz.github.io/playhub/`.
4. The browser connects directly to Supabase from there, using the URLs allowed in Supabase's URL Configuration.

---

7. Design decisions and why

Decision	Why
Supabase for accounts and data	Secure password storage, sessions, and a database without writing a backend.
Login handled in the browser	Works on static hosting such as GitHub Pages, which can't run server code.
RLS policies instead of a server	Rules enforced in the database can't be bypassed by changing the browser code.
Trigger for profiles	Guarantees every new user gets a profile, with no extra code in the app.
Separate `.js` files	Easier to read, edit, and debug than one long file.
Immediately invoked functions in JS	Prevents variables from colliding between files.
Custom events between `auth.js` and `game.js`	Keeps login logic and game logic separate.
Flask for local development	The standard lightweight Python web server.
`.env` for keys	Keeps secrets out of code.
`build_static.py`	GitHub Pages can't run Flask, so the page is pre-built into plain files.
`.nojekyll`	Stops GitHub from running Jekyll, which would hide some files.
`gunicorn` on Render	Production-grade server.
Plain DOM for the game	No external game engine needed, so there's less to install and break.
Progress bar floor of 1	Prevents division by zero when everyone has 0 points.
`textContent` over `innerHTML`	Prevents code injection through usernames.

8. Known limitations

1. **Scores can be cheated.** The browser sends the new totals, so someone with developer tools could write any number. **Fix:** move the score calculation into a Supabase database function (RPC) that checks the round length on the server.
2. **Two simultaneous saves can overwrite each other.** Read-then-write is not atomic. **Fix:** use a single SQL statement `points = points + $1` in a database function.
3. **Leaderboard shows only 50 players. Fix:** add paging or a "show more" button.
4. **Anon key is visible** in the page source. This is expected for Supabase. Protection depends on RLS being correct.
5. **`docs/` is a snapshot.** Editing templates or static files has no effect on the live site until `build_static.py` runs again.
6. **Free Render needs a card** and sleeps after inactivity. GitHub Pages avoids both issues.
7. **No password reset link** is built into the UI. Supabase can send reset emails, but a reset page has not been added yet.
8. **Timer runs in the browser.** Refreshing mid-round ends the round without saving.

---

9. Troubleshooting: every error we hit and its cause

Error or symptom	Cause	Fix we applied
`TemplateNotFound: base.html`	`base.html` was missing, or in the wrong folder, or named `base.html.txt`.	Created `templates/base.html` next to `app.py`.
`static` folder inside `templates`	Folder created in the wrong place.	Moved `static` up one level to the project root.
`Invalid path specified in request URL` on signup	`SUPABASE_URL` ended with `/rest/v1/`.	Removed that suffix from `.env`.
`ModuleNotFoundError: No module named 'dotenv'`	Packages not installed in the venv.	`pip install -r requirements.txt`.
`'origin' does not appear to be a git repository`	The remote was never added. The script printed the command but didn't run it.	`git remote add origin ...`.
`403 Permission denied` on push	Git was signed in as a different account.	Signed in as `zzanizzzz`.
Render asks for a card	Render's free tier changed to require billing details.	Switched to GitHub Pages.
Pages shows nothing	`docs/` folder didn't exist yet.	Ran `build_static.py` and pushed `docs/`.
`No module named ...` after activating venv	Venv activated but packages not installed.	Installed requirements inside the venv.

---

10. Command reference

Activate the virtual environment (Windows PowerShell):

```powershell
venv\Scripts\activate
```

Install packages:

```powershell
pip install -r requirements.txt
```

Run the local site:

```powershell
python app.py
```

Then open `http://127.0.0.1:5000`.

Set up files and git:

```powershell
python setup_deploy.py
python setup_deploy.py --git
```

Build the GitHub Pages copy:

```powershell
python build_static.py
```

Check what Git will upload:

```powershell
git status
```

Publish a change:

```powershell
python build_static.py
git add .
git commit -m "describe your change"
git push
```

Connect to GitHub (once):

```powershell
```

```
git remote add origin https://github.com/zzanizzzz/playhub.git
git push -u origin main
```

Check the remote address:

```powershell
git remote -v
```

Final notes

- Never share your `.env` file or paste your keys into a chat.
- Run `build_static.py` every time you change any file that the live site uses.
- To improve scoring security, move score updates into a Supabase database function (see Known limitations).